

Machine Learning - Introduction

Dr. K. K. THYAGHARAJAN
Professor & Dean (Research)
R.M.D. Engineering College

Scan the QR code and it will provide the link to all videos. It is a trusted link and safe.



<https://www.youtube.com/c/KKTVirtualLearning>

UNIT II INTRODUCTION TO MACHINE LEARNING

9

Introduction to Machine Learning - Types of Machine Learning - Supervised and Unsupervised - Data exploration - Target Variables, Independent Numerical Variables, Categorical Variables - Splitting of Data - ~~Feature Scaling of Data.~~

Machine Learning - Introduction

Definition:

Machine learning is indeed a subset of artificial intelligence (AI) that develops algorithms and models enabling computers to **learn patterns** from data and predict outputs for new inputs. It can also be used to make decisions based on the learned patterns from the given data.

Machine learning **algorithms improve and optimize their performance** over time as they are exposed to more data, without being explicitly programmed for specific tasks.

To develop predictive models, machine learning brings together statistics and computer science. The types of machine learning are given below.

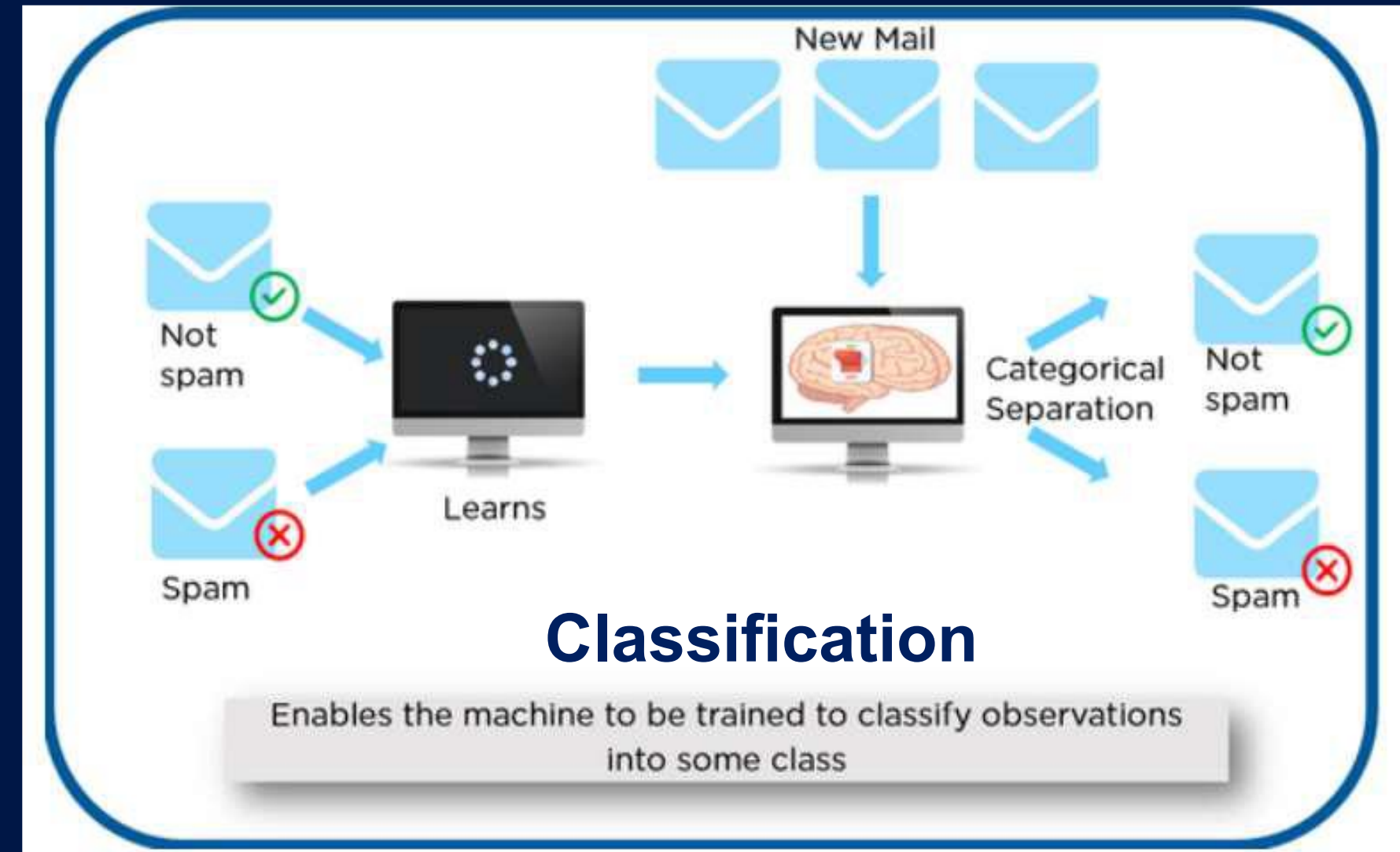
1. Supervised learning - spam / not spam
 - i) Regression - car price prediction
 - ii) Classification - diabetic prediction
2. Unsupervised learning
 - i) Clustering - cats and dogs
 - ii) Association - if bread is bought then butter
3. Semi-supervised learning
4. Reinforcement learning

Supervised Learning

Supervised Learning:

Supervised learning is a type of machine learning where a computer learns from **labeled data** to make predictions. Labeled data means each example has an **input** (like an image or a number) and a **correct output** (like a category or a value). Learning happens through training.

Such an algorithm uses data to learn patterns and uncover relationships between other parts of the given dataset and the target.



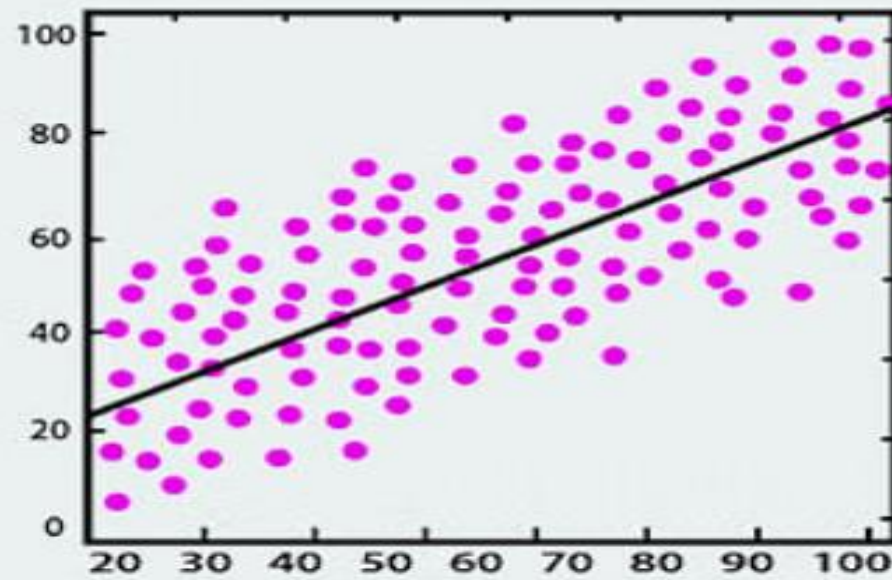
Types of Supervised Learning:

- ❖ **Regression:** A problem may use regression when the output is a **real value (number)**. For example, variable such as dollars, weight, etc. has **real values (numbers)**.
- ❖ **Classification:** A classification problem is when the output is a **categorical value**. For example, the variable color has categorical values such as red or green or blue. Variable e-mail can have categorical values 'spam' or 'not spam', classifying the **handwritten numbers as odd or even**, etc.

Regression finds correlations between dependent and independent variables. For example, if the regression equation is found to be $y = mx$, then **x** is an **independent variable** and **y** is a **dependent variable**. The Regression algorithm's task is finding a mapping function so that the input variable "x" is mapped to the continuous output variable "y." Therefore, regression algorithms help **predict continuous variables** such as house prices, market trends, weather patterns, oil and gas prices (a critical task these days!), etc.

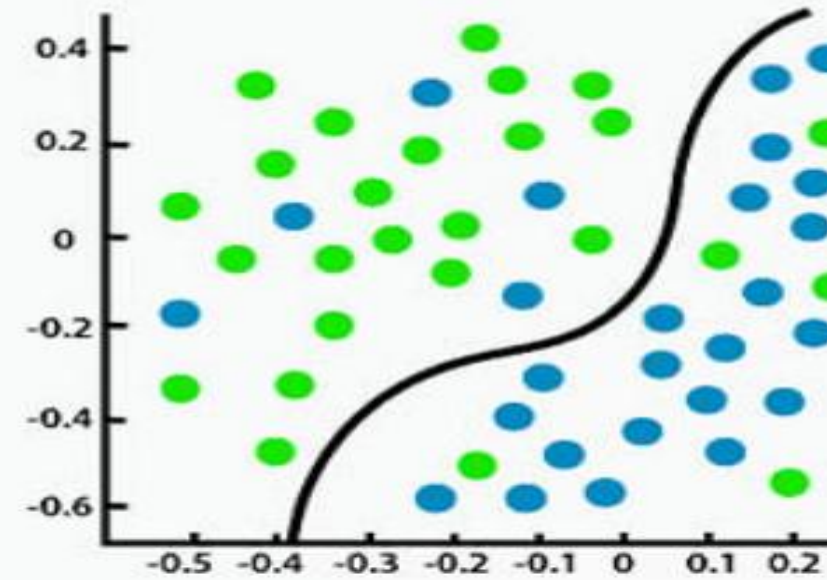
Classification is an algorithm that finds functions to **divide the dataset into classes (Categories)** based on various parameters. When using a Classification algorithm, a computer program gets taught on the training dataset and categorizes the data into various categories depending on what it learned. Classification algorithms find the mapping function to **map an input (x) to any one of the discrete outputs (Y1,Y2,Y3 etc.)**. The algorithms estimate **discrete values** (in other words, binary values such as 0 and 1, yes and no, true or false, based on a particular set of independent variables. To put it another, more straightforward way, classification algorithms predict an event occurrence probability by fitting data to a logit function. Classification algorithms are used for things like **email and spam classification**, predicting the willingness of bank customers to pay their loans, and **diabetic / non-diabetic prediction**.

Supervised Learning



Regression

versus



Classification

Advantages:

- Since supervised learning works with the labeled dataset we can have an exact idea about the classes of objects
- These algorithms help predict the output based on prior experience

Disadvantages

- These algorithms are not able to solve complex tasks
- It may predict the wrong output if the test data is different from the training data
- It requires lots of computational time to train the algorithm

Supervised Learning

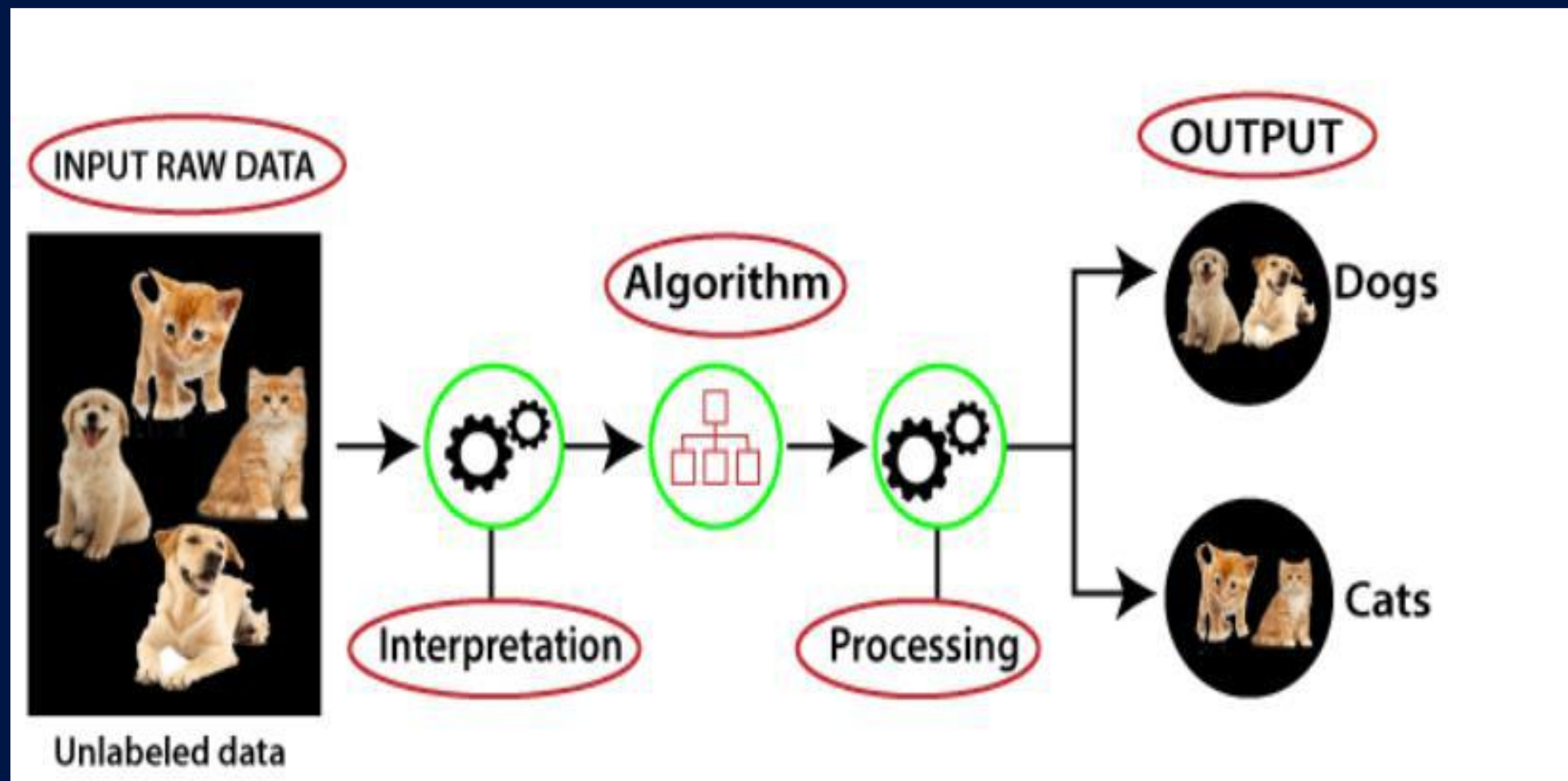
iii. Applications of Supervised Learning

- ❖ Supervised Learning algorithms are used in **image segmentation**. In this process, image classification is performed on different image data with pre-defined labels.
- ❖ Supervised algorithms are also used in the **medical field for diagnosis** purposes. It is done by training the system using medical images with labels of past disease conditions. After training, the machine can identify a disease for the new patients.
- ❖ Supervised Learning classification algorithms are used for identifying **fraud transactions**, fraud customers, etc. It is done by using historical data to identify the patterns that can lead to possible fraud.
- ❖ In **spam detection filtering**, classification algorithms are used. These algorithms classify an email as spam or not spam. The spam emails are sent to the spam folder.
- ❖ Supervised learning algorithms are also used in **speech recognition**. The algorithm is trained with voice data, and various applications such as voice-activated passwords, voice commands, etc are implemented

Unsupervised Learning

Unsupervised Learning:

In unsupervised learning, we provide **unlabeled training data**. It means that we provide input data but we don't provide the corresponding output-label or output-class. So, there is no explicit guidance or training. The algorithm is finding patterns, relationships, or structures within the input data and group the inputs based on that.



In the figure shown, unlabeled pictures are given as input.

The patterns, structures and the relationships are extracted and interpreted. We apply K-means clustering algorithm or any clustering algorithm.

This algorithm divides the data objects into groups or clusters according to the similarities and difference between the objects.

Unsupervised Learning

Types of Unsupervised Learning

a) Clustering:

Clustering is a **method of grouping** the objects such that the objects with **most similarities** remains into a group, and objects that have **less or no similarities** into another group. Some of the popular clustering algorithms are given below:

- ❖ **K-Means** Clustering algorithm
- ❖ **Principal Component Analysis** (PCA) is primarily used for dimensionality reduction and works without labeled data. It identifies patterns and structures in the data by finding the directions (principal components) that maximize variance.
- ❖ **DBSCAN Algorithm** (Density-Based Spatial Clustering of Applications with Noise): It groups data points that are closely packed together (based on a defined distance metric) while marking points in low-density regions as noise or outliers.

Unsupervised Learning

b) Association:

This is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. E.g. People **who buy X (bread) also tend to buy Y (butter or jam)**.

This algorithm is mainly applied in **Market Basket analysis**, Web usage mining, continuous production, etc. Some popular algorithms of Association rule learning are Apriori Algorithm, Eclat, FP-growth algorithm.

Below is the list of some popular unsupervised learning algorithms:

K-means clustering

Decision Tree

KNN (k-nearest neighbors)

Hierarchical clustering

Anomaly detection

Neural Networks

Principal Component Analysis

Independent Component Analysis

Apriori algorithm

Singular value decomposition

Unsupervised Learning

Advantages:

- These algorithms can be used for **complicated tasks** compared to the supervised ones because these algorithms work on the unlabelled dataset
- Unsupervised algorithms are preferable for various tasks as **getting the unlabelled dataset is easier** as compared to the labelled dataset

Disadvantages:

- The output of an unsupervised algorithm can be **less accurate** as the dataset is not labelled, and algorithms are **not trained with the exact output** in the prior
- **Working** with Unsupervised learning **is more difficult** as it works with the unlabelled dataset that does not map with the output

Unsupervised Learning

Applications of Unsupervised Learning

- **Network Analysis:** Unsupervised learning is used for identifying **plagiarism and copyright** in document network analysis of text data for scholarly articles
- **Recommendation Systems:** **Recommendation systems** widely use unsupervised learning techniques for building recommendation applications for different web applications and e-commerce websites
- **Anomaly Detection:** Anomaly detection is a popular application of unsupervised learning, which can identify unusual data points within the dataset. It is used to discover **fraudulent transactions**
- **Singular Value Decomposition:** Singular Value Decomposition or SVD is used to **extract particular information** from the database. For example, extracting information about each user located at a particular location

Semi-supervised Learning

Semi-supervised Learning:

Generally, during semi-supervised machine learning, algorithms are first fed with a small amount of labeled data similar to supervised learning model until the model gives accurate results.

In the next step, large number of unlabeled data is fed to a clustering algorithm which creates clusters with pseudo labels. Here the (clustering) accuracy will be less.

Now, the labels of labeled training (supervised) and the pseudo labels created in unsupervised learning are linked together.

The input data in labeled training data and unlabeled training data are also linked.

At the end, again train the model with the new combined input as did in the first step. It will reduce errors and improve the accuracy of the model.

Semi-supervised Learning

Semi-supervised machine learning is often employed to train algorithms for **classification and prediction** purposes if large volumes of labelled data are unavailable.

For example, an algorithm may first be trained with a smaller quantity of labeled speech data and then be fed a much larger set of unlabeled speech data to create a machine-learning model capable of **speech recognition**.

Assumptions followed by Semi-Supervised Learning:

To understand the relationship between the unlabeled objects, the following assumptions are used

Continuity assumption:

The objects near each other tend to share the same label or falls into the same group.

Cluster assumptions:

The points in the same cluster share the output label.

Manifold assumptions:

This assumption helps to use distances and densities, and this data lie on a manifold of fewer dimensions than input space.

Reinforcement Learning

Reinforcement Learning:

Reinforcement learning is a **feedback-based learning** method, in which a **learning agent** gets a **reward** for each right action and gets a **penalty** for each wrong action. The agent learns automatically with these feedbacks and improves its performance.

The agent interacts with the environment and explores it. **The goal of an agent is to get the most reward points**, and hence, it improves its performance.

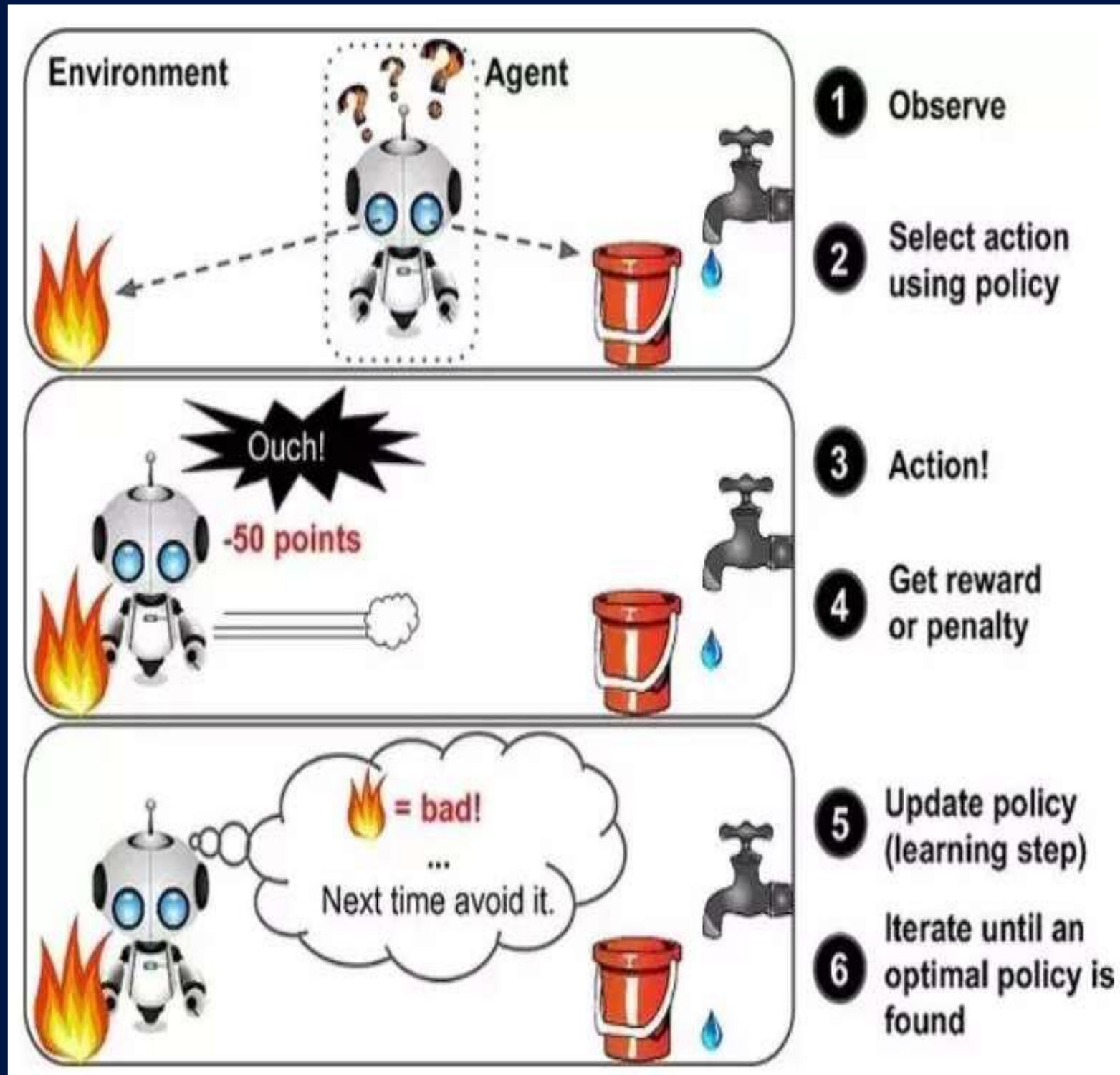
For instance, an algorithm may be optimized by playing successive games of chess, which allows it to learn from its past successes and failures playing each game.

The **robotic dog**, which automatically learns its movement, is another example of Reinforcement learning.

Reinforcement learning is often used to create algorithms that must effectively make **sequences of decisions or actions** to achieve their aims, such as playing a **game** or **summarizing an entire text**.

Reinforcement Learning

Example 1



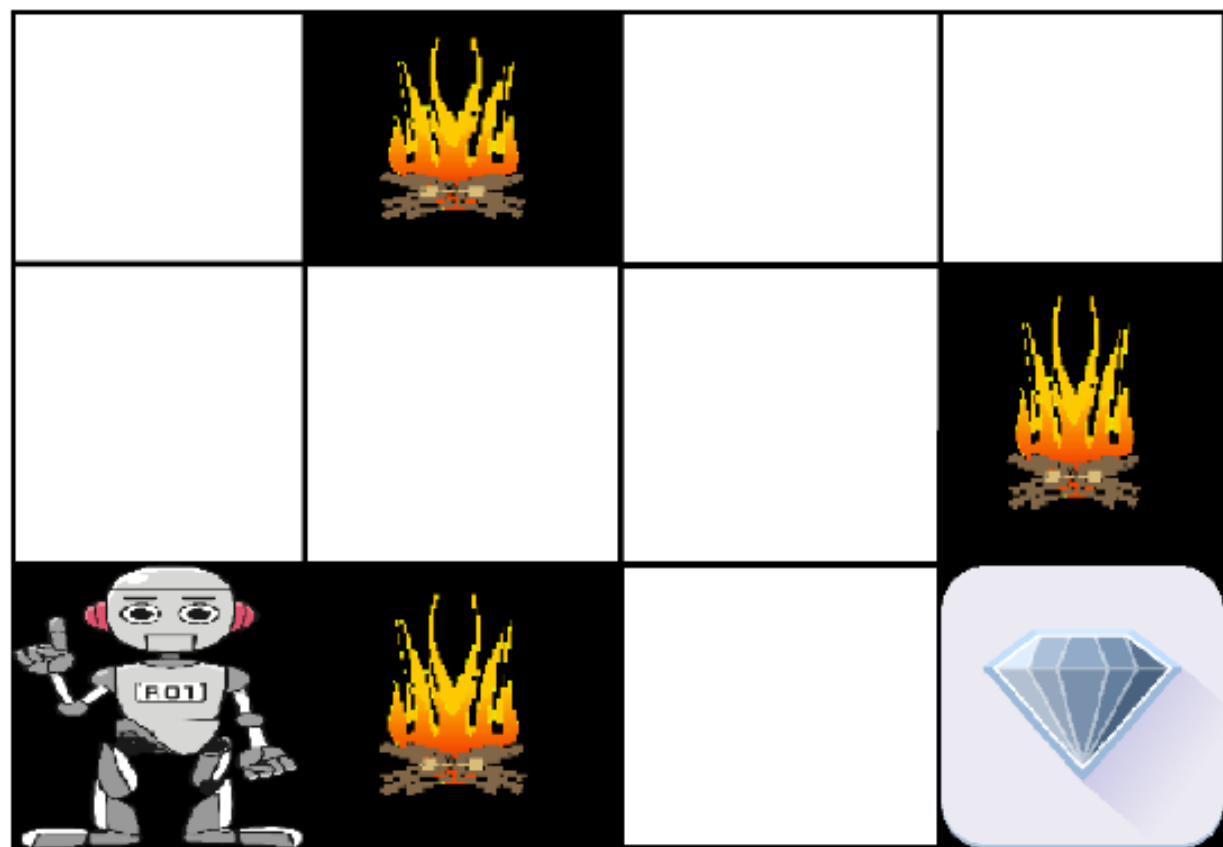
In the figure, the agent is given 2 options i.e. a path with water or a path with fire.

In the reinforcement algorithm, if the agent uses the path with fire then the rewards are subtracted and the agent tries to learn that it should avoid the fire path.

If it had chosen the water path or the safe path then some points would have been added to the reward points.

The agent then would try to learn what path is safe and what path isn't. It is leveraging the rewards obtained, the agent improves its environment knowledge to select the next action.

An example of an algorithm used in reinforcement learning is the Markov Decision Process.



Input: The input should be an initial state from which the model will start.

Output: There are many possible outputs as there are a variety of solutions to a particular problem.

Training: The training is based upon the input; The model will return a state and the user will decide to reward or punish the model based on its output.

The model keeps continues to learn. The best solution is decided based on the maximum reward.

Data Exploration

Data sources can be databases, excel files (.CSV), websites etc. (raw data).

Raw data cannot be directly used for model building and analysis as it will be inconsistent and not suitable for prediction. It has to be treated for anomalies and missing values.

There are discrepancies like Nan/ Null / NA values in many rows and columns.

Variables can be of different types such as character, numeric, categorical, and continuous. Identifying the predictor and target variable is also a key step in model building.

Target is the dependent variable and predictor is the independent variable based

Categorical or discrete variables are those that cannot be mathematically manipulated.

Continuous variables can be interpreted using mathematical functions like finding the average or sum of all values.

Data exploration refers to the initial step in data analysis

In data exploration, data visualization and statistical techniques are used to describe and understand the dataset

It includes both manual analysis and automated data exploration software solutions that visually explore and identify relationships between different data variables, the structure of the dataset, the presence of outliers, and the distribution of data values to reveal patterns and relationships between data

Various steps involved:

- > Import the required libraries
- > Read / Load data
- > Identify variables
- > Variable analysis
- > Handling missing values
- > Handling outliers
- > Feature engineering

When you develop a machine learning model, after some point of time, you'll realize that you are struggling to improve the model's accuracy. In such a situation, data exploration techniques will come to your rescue.

Remember that the quality of the inputs to the model decides the quality of the model's output

Spend significant time on exploration and analyzing data.

Generally, data exploration, cleaning and preparation can take up to 70% of the total project time

First, identify the target or dependent variables and identify the independent variables.

Refer II Unit Digital Notes & Include Pandas functions `df.info()`, `df.describe()` etc from I Unit

Three Types of Variables

Aid to Memory

Output variable – dependent variable - predicted variable - target variable - response variable - left hand side variable - effect - outcome variable. (OD PTR LEO)

Input variable – independent variable – regular variable – cause - right hand side variable - explanatory variable - predictor variable. (I²R C REP)

Categorical Variable – Qualitative variable – Nominal Variable

If the variable animal (categorical) may have value 'cat' (label) - **labeled target.**

	<u>city</u>	Experience	Salary
0	Salt Lake City	10	120
1	Seattle	5	120
2	San Francisco	5	140
3	Seattle	3	100
4	Seattle	1	70
5	San Francisco	2	100
6	Salt Lake City	1	60
7	San Francisco	2	110
8	Seattle	4	100
9	Salt Lake City	2	70

1. Target Variables

The **target variable** is the variable that the user would **want to predict** using the given dataset.

The **dependent variable** is assigned a role as the **target variable**, while an **independent variable (predictor variable)** may be given a role as the **regular variable**.

So, a **target variable** is a variable whose values are modeled and predicted by other variables.

A **predictor variable** is a variable whose values will be used to predict the value of the target variable.

In the case of regression models, the **target is real-valued**, i.e. value is in real numbers. In comparison, with a classification model, the target is binary or multivalued (discrete levels).

Target variables may vary depending on the goal and available data. The model's outcomes will be nothing if the target isn't well understood. If an animal image is given as input and if it is to be identified by the model as 'cat', the output variable (e.g. animal) should be assigned a value 'cat' i.e. label. This called **labelled target**.

In the absence of a **labeled target**, supervised machine learning algorithms would not be able to map available data to outcomes.

A child would be incapable of figuring out that dogs are called dogs without being told a few times. Well-defined targets are important, as the only thing the algorithm does is learn a function that maps the relationship between input data and the target.

Determining the target variable can require running an existing suboptimal system until enough training data is collected.

Targets are often manually labeled in a dataset, but there are ways to automate this process

A **dependent variable** is a variable that changes when the independent variable value changes. It's the outcome you're interested in measuring, and it "depends" on your independent variable. In machine learning it is called the **target variable**.

In **statistics**, dependent variables are also called:

- ❖ **Response variables** (they respond to a change in another variable)
- ❖ **Outcome variables** (they represent the outcome you want to measure)
- ❖ **Left-hand-side variables** (they appear on the left-hand side of a regression equation)

The dependent variable is what you record after you've manipulated the independent variable. You use this to check whether and to what extent your independent variable influences the dependent variable by conducting statistical analyses.

Based on your findings, you can estimate the degree to which your independent variable variation drives changes in your dependent variable.

You can also predict how much your dependent variable will change as a result of variation in the independent variable.

2. Independent variables:

An independent variable is a variable you manipulate or vary in an experimental study to explore its effects. It's called "independent" because it's not influenced by any other variables in the study.

The **independent variable is the cause**. The dependent variable is the **effect**. Its value depends on changes in the independent variable.

Independent variables are also called:

- ❖ **Explanatory variables** (they explain an event or outcome)
- ❖ **Predictor variables** (they can be used to predict the value of a dependent variable)
- ❖ **Right-hand-side variables** (they appear on the right-hand side of a regression equation).

There are two main types of independent variables.

Experimental independent variables: These variables can be directly manipulated.

Subject variables: These variables cannot be manipulated, but they can be used to group the subjects categorically.

Experimental variables:

In experiments, you manipulate independent variables directly to see how they affect your dependent variable. The independent variable is usually applied at different levels to see how the outcomes differ.

You can apply just two levels to find out if an independent variable has an effect at all.

You can also apply multiple levels to find out how the independent variable affects the dependent variable.

Subject variables:

Subject variables are characteristics that vary across participants, and they can't be manipulated. For example, **gender, race, height, income, and education** are all important subject variables that are treated as independent variables. It's not possible to randomly assign these to participants.

3. Categorical Variables

In machine learning, it is common to encounter categorical data (values) from variables such as **gender (Male, Female)**, race, nationality or occupation.

Categorical data will be independent when they are given as input to the model and they may be dependent when they are output variables.

One of the main challenges of working with categorical data is that most machine learning algorithms are designed to work with numerical data. This means that **categorical data (values)** must be transformed into a numerical format.

Categorical (or Nominal or Qualitative) **Variables** will be names

Categorical **values** will also be names but they are called Labels

The Input and output values may also be Categorical values but they are converted to numeric values in the model.

Categorical Data Encoding

Categorical Data Encoding:

a) Replacing with Numbers

It is the process of replacing a categorical variable with a numerical value. For example, let us take two categorical data Female and Male. These data can be replaced or represented by numerical values 0 and 1 respectively.

Convert this to numerical values using the replace method of Pandas data frame

Index	Gender (Category)
0	Male
1	Female

`df.replace({'Female': 0, 'Male': 1})`

This will assign the numerical value '0' whenever it encounters 'Female' in the dataset and assign '1' for 'Male' as shown below. Note that the index values of the data are not changed.

Index	Gender (Category)
0	1
1	0

Categorical Data Encoding

The ‘replace’ method might be a suitable approach for a small number of categories, but it can become impractical when dealing with many categories.

When the categories are more, another method called **Label Encoder** may be used for encoding categorical variables (i.e. values). It assigns a unique numerical value to each category in the categorical variable. For the above example, it produces the same result. The python code for implementing this is given below.

```
from sklearn import preprocessing
le = preprocessing.LabelEncoder()
le.fit(df['Gender'])
df['Gender'] = le.transform(df['Gender'])
```

In this code, Gender is the column name i.e. name of the categorical variable. This variable may have values ‘Male’ or ‘Female’. This column is in the data frame named ‘df’.

output :

Index	Gender (Category)
0	1
1	0

Categorical Data Encoding

b) One Hot Encoding / Dummyifying

It is the process of creating a new binary column for each category in a categorical variable, with a 0 or 1 indicating the presence or absence of that category. For example, consider the following Pandas data frame

Index	Color (Category)
0	Red
1	Green
2	Blue

Categorical Variable Color

Index	Color_Red	Color_Green	Color_Blue
0	1	0	0
1	0	1	0
2	0	0	1

One-hot encoding of Categorical Variable Color

In this method, three dummy variables are created automatically by the code. The names use the column name (variable name) as the first part and the value or data as the second part by connecting them using an underscore as shown in the first row of the above table.

if the category is 'Red', numerical value 1 is put under the column 'Color_Red', and 0 is put in all other columns in that row. This is also called 'one **hot encoding**' or **dummyifying**. "dummify" is a more colloquial term, and "**One Hot encoding**" is the technical term used in the machine learning literature.

Categorical Data Encoding

This can be implemented using `get_dummies` function available in Pandas

```
import pandas as pd
```

```
X_encoded = pd.get_dummies(df, columns=['Color'])
```

It can also be implemented using the `OneHotEncoder` class provided in `sklearn` package of Python

```
from sklearn.preprocessing import OneHotEncoder
```

```
enc = OneHotEncoder()
```

```
enc.fit(df)
```

```
X_encoded = enc.transform(df).toarray()
```

Categorical Data Encoding

Advantages and Disadvantages of Encoding Methods

Dummies (one hot encoding) create separate columns for each category, allowing the model to learn the relationships between the individual categories and the target variable. The model can learn more complex relationships between the categories and the target variable.

There are certain situations in which Dummies may not be the best approach. Here are the most important ones:

High cardinality (no. of .distinct classes): One Hot Encoding creates a separate column for each category in the categorical variable. This can lead to a **high number of columns**, especially if the categorical variable has many unique values. In such cases, One Hot Encoding may result in a sparse and bulky data set, which can be challenging to work with.

Memory constraints: One Hot Encoding can also be problematic if the data set is large and requires a **lot of memory** to store. The resulting data set can take up a lot of space, which may not be feasible if memory is limited.

Categorical Data Encoding

Multicollinearity: Occurs when there is a high correlation between the **dummy variables**, which can cause the coefficients in the model to be unstable and difficult to interpret. **Dummy variables are naturally correlated because they are created from the same categorical variable.**

In these situations, alternative encoding methods, such as **label encoder** or **target encoding**, may be more appropriate, which can handle high cardinality more efficiently.

Replacing for numbers and label encoder, on the other hand, implies an **ordinal relationship** between the categories and does not create separate columns for each category, which **can mislead** the results if the categories do not have an inherent order.

For example, suppose you replace “Red” with 1, “Green” with 2, and “Blue” with 3 in your dataset. In that case, your model may assume that “Green” is twice as important as “Red” and “Blue” is three times as important as “Red”. This can introduce **bias** into the model, as it makes assumptions about the order in which you assign the numbers. It is similar to assigning weightage for each categorical data. These two methods allow the model to learn the overall relationship between the numerical value and the target variable, which may not capture the full complexity of the data.

Categorical Data Encoding

c) Frequency Encoding

This method gives each category a number based on how often it appears in the data set. For example, if we have lots of dogs and only a few cats and birds on our dataset, then dogs will get a higher number. The numbers add up to 1, representing 100% of animals in the data set.

Animal Type	Frequency Encoding
Dog	0.6
Cat	0.3
Bird	0.1

This is useful if we are interested in how many times a particular animal category appears. But if we are more interested in how much it costs to care for different animals, then target encoding will be useful.

Categorical Data Encoding

d) Target Encoding

Target encoding involves replacing a categorical feature with the average target value of all data points belonging to the category.

Consider the data frame shown in Figure 1, here APPLE and MANGO are categorical values. Fruits is a categorical variable (Column name is the variable). These two categorical values may be encoded by using any one of the above said 3 methods. YEAR 1 and YEAR 2 columns give the prices. If our target is price, then the categorical values may be replaced by the average prices as shown in Figure 2. For example, APPLE may be replaced by $((80+100)/2 = 90)$ and so on. This method is called target encoding.

FRUITS	YEAR 1	YEAR 2
APPLE	80	100
MANGO	50	100

Figure 1

FRUITS	YEAR 1	YEAR 2
90	80	100
75	50	100

Figure 2

One of the challenges with Target Encoding is **overfitting**. This happens when the model learns the training data too well, **including its noise and outliers** and performs well on the training data but does not generalize well to new data. This may happen when a few values are very high or very low.

Another challenge with the target encoding is **the Leakage of the target variable** into one of the features. This is due to some larger values in the data. In these cases, the model with target encoding does not generalize well to new data.

There are several techniques to **reduce overfitting with target encoding**, here are two of the popular techniques.

i) **Leave One Out Target Encoding** involves taking the mean target value of all data points in the category except the current row.

ii) **Leave One Fold (group) Out Target Encoding** involves taking the mean target values from data points in all other folds except the data in the current fold.

Both these techniques ensure that every time a particular category appears we don't end up with the same value.

Splitting of Data (Training, Validation and Test Data)

Data is divided into three sets — **Training, validation** and **Test**.

The **training set** is fed to the learning algorithm in a loop teaching about the hidden features / patterns in the data for understanding the relationship and improving the prediction.

The validation set as the name suggests is used to validate (evaluate) the model performance. This step helps in minimizing the error and gives a chance to tune the hyperparameters (**learning rate etc.**) and configurations a little more (e.g. degree of polynomial in regression).

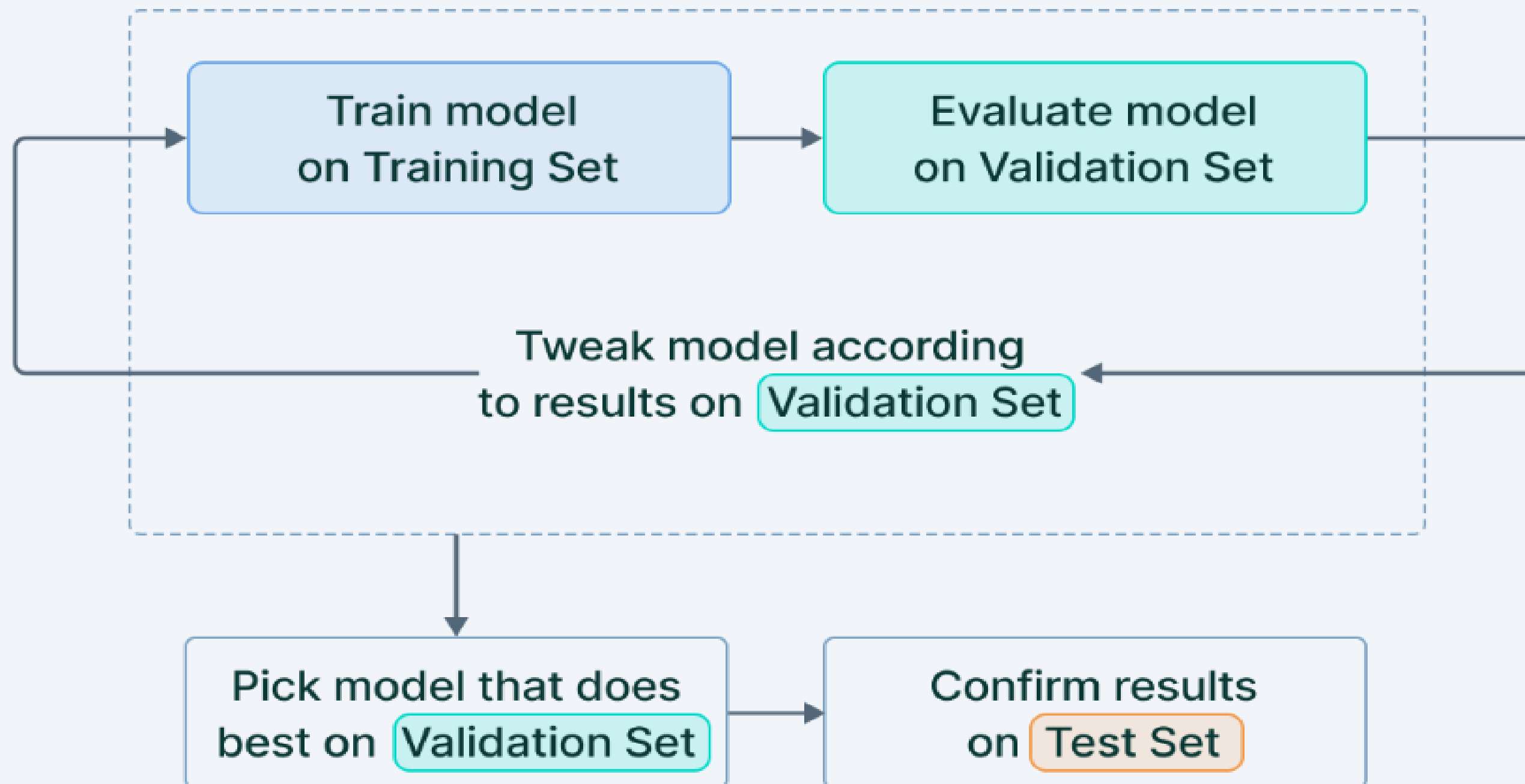
One of the benefits of the validation is that it **prevents overfitting** (Overfitting- a term used to describe not being able to make accurate predictions on the data not seen before) and you can say that the model is **generalized** if a new unknown data is classified correctly.

The **test set** is used to test the model after completing the training, providing an accurate, unbiased model performance.

Usually 60–20–20 percentage policy is followed i.e. 60% goes to the Training set, 20 % to validation set and remaining on the test set.

You can also use 80% data for training and 20% data for testing. This ratio depends on the problem and the model used.

Training data/validation/test



Tweaking a model after validation refers to modifying or fine-tuning the model based on the insights or results observed during the validation process. This can involve changes to the model's architecture, hyperparameters, training procedure, or data preprocessing techniques to improve performance on unseen data.

Refer Digital notes also

Feature Scaling

Feature Scaling Methods:

a) Normalization: This is the simplest method of scaling where the features are rescaled to a given range. It is of two types

i) Min-Max Normalization: Also called as Min-Max Scale, this transforms data i.e. each feature individually to fall **between zero and one**. This method **preserves the shape of the original distribution**. The transformation is given by the formula.

$z = (x - \min(x)) / (\max(x) - \min(x))$ where z is the normalized value.

Example: An algorithm that uses K-nearest neighbors (KNN).

Python code:

```
scaler = MinMaxScaler()  
x_minmax = scaler.fit_transform(x)
```

ii) Robust Scaler: As per sklearn, this type of normalization transforms the vector by **subtracting the median** and scales the data between the **1st quartile (25% value) and 3rd quartile (75% value)**.

Python code for robust scaling:

```
from sklearn.preprocessing import RobustScaler  
transformer = RobustScaler().fit(X)  
transformer.transform(X)
```

Feature Scaling

b) Standardization: This is the most common form of feature scaling applied to many learning algorithms. First, we calculate the distribution mean and standard deviation for each feature and calculate the new data point by using the formula.

$$x' = \frac{x - \bar{x}}{\sigma}$$

Here, σ is the standard deviation of the feature vector, and $\bar{x}$ is the average of the feature vector.

Standardization results in a distribution that has a standard deviation of 1 and a mean of 0. Examples of algorithms that use Standardization are Support Vector Machine (SVM), logistics regression and neural networks. The Python code to implement this scaling is given below

```
from sklearn.preprocessing import StandardScaler  
sc = StandardScaler()  
X_train[:, :] = sc.fit_transform(X_train[:, :])
```

Feature Scaling

Where to use Feature Scaling?

Min-Max Scaler is used to have a **light touch** on features

Robust Scaler is used to reduce the **influence of outliers**.

Normalization is used when the distribution of data **does not follow Gaussian distribution** like neural networks.

Standardization is applied when **Gaussian distribution is followed** e.g. Principal Component Analysis (PCA).

For **K-Means Clustering**, the Euclidean distance is important, so **Feature Scaling** makes a huge impact.

Feature scaling also makes a huge impact on any algorithms that rely on **gradients**, such as linear models that are fitted by **minimizing loss with Gradient Descent**.

Some feature selection methods will depend on the scale of the data, in which case it seems best to scale beforehand.

A **tree-based model won't suffer from unscaled data**, because scale doesn't affect them at all.

Disadvantage: Both normalization and standardization are **sensitive to outliers**.

Feature Scaling

Why is feature scaling needed?

Feature scaling is done during **Data Preprocessing** to adjust the features to a common scale.

Feature scaling is similar to the database normalization method and is used to normalize the range of **independent** (features) data.

Feature scaling helps in not drifting towards high-range input values i.e. it resolves (or) reduces the disparity between the features of data like **high bias**, and values varying from each other on a broad scale (high values).

It brings the value / magnitude of the numbers close to each other within a range for easy comparison. This makes the models interpret the features easier, bring them into the same scale and make them perform better.

Generally, various **Machine Learning models don't generalize** well on data when the data has **high-scale variance**. The same data can be bad for one model, but great for another.

You can have the best model crafted for any sort of problem. But if you feed it garbage, it'll spew out garbage. It's worth noting that **"garbage" doesn't refer to random data**. It's a **harsh label** we attach to any data that doesn't allow the model to do its best.

Feature Scaling

Performing Data Scaling Before Data Split

Disadvantage:

Performing feature scaling before dataset splitting can introduce **data leakage (from test data)**, as the scaling parameters such as mean and standard deviation are influenced by the test set, leading to over optimistic evaluation of results.

Data leakage is a phenomenon that occurs when **information outside of the training data is used to train** a model. It essentially violates the independence of the training data and allows it to be altered by information from test data.

The model's performance would only be limited to the range in which the data is scaled e.g. in the range of $[0,1]$. So, the data in different ranges (test data) will make the model fail.

Advantage:

Scaling the entire dataset before splitting **ensures that the same scaling factors** are applied to both the training and test sets. This **leads to consistency** in the data preprocessing pipeline. It allows you to explore the entire dataset during exploratory data analysis (EDA).

Feature Scaling

Performing Data Scaling After Data Split:

Feature scaling is performed only on the training set. This ensures that there is no **data leakage** from the **test set**.

Advantage:

Parameters obtained during the normalization/scaling of training data can be used to normalize the test data and also change it back to the original scale when showing the output/predictions.

This approach provides a more realistic evaluation of the model's performance since it mimics real-world scenarios where new, unseen data requires scaling.

Disadvantages

It may cause a slight shift in the distribution of the training set features, potentially leading to a decrease in model performance.

THANK YOU

THANK YOU

How to Install Python on Your Laptop?

(Activity / Mini Project)

Dr. K. K. THYAGHARAJAN
Professor & Dean (Research)
R.M.D. Engineering College



Flynn's Classification of Multiprocessor Organization

R.M.D. Engineering College

Dr. K. K. THYAGHARAJAN
Professor & Dean (Research)

